

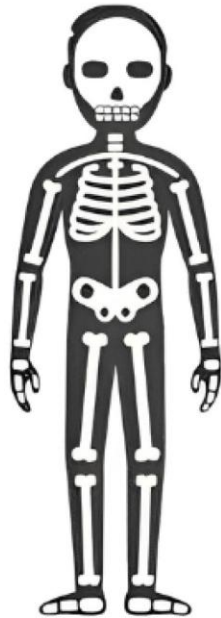
CSS



HTML



HTML the Skeleton



CSS



CSS the Skin



JS



JavaScript the Brain



WHAT WE WILL DISCUSS ?

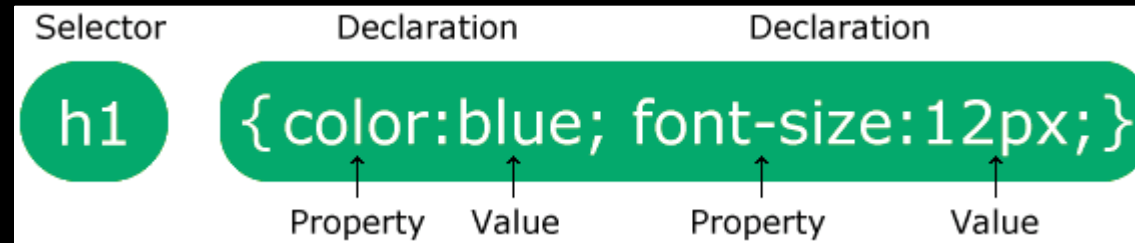
- What is CSS ?
- How to Add CSS
- CSS Selectors
- CSS Comments
- CSS Colors
- CSS Text
- CSS Backgrounds
- CSS Borders
- CSS Margin and Padding
- CSS Box Model
- CSS Outline
- CSS Display Property
- CSS Flexbox
- CSS Grid
- CSS Position
- CSS Lists
- CSS Tables
- CSS Units
- CSS Pseudo-Classes
- CSS Pseudo-Elements
- CSS Overflow
- CSS Z-Index & Float
- CSS Transitions
- CSS Animations
- CSS Media Queries
- CSS Specificity

WHAT IS CSS ?

CSS is the language we use to style a Web page.

- CSS stands for Cascading Style Sheets
- CSS describes how HTML elements are to be displayed on screen, paper, or in other media
- CSS saves a lot of work. It can control the layout of multiple web pages all at once
- External stylesheets are stored in CSS files

CSS Syntax :



HOW TO ADD CSS

External CSS :

```
<head>
<link rel="stylesheet" href="mystyle.css">
</head>
```

Internal CSS :

```
<style>
body {
  background-color: linen;
}
</style>
```

Inline CSS :

```
<h1 style="color:blue;text-align:center;">This is a heading</h1>
<p style="color:red;">This is a paragraph.</p>
```

CSS SELECTORS

CSS Element Selector:

```
p {  
  text-align: center;  
  color: red;  
}
```

CSS ID Selector :

```
#para1 {  
  text-align: center;  
  color: red;  
}
```

CSS Class Selector :

```
.center {  
  text-align: center;  
  color: red;  
}
```

CSS COMMENTS

```
/* This is a single-line comment */  
p {  
  color: red;  
}
```

CSS ERRORS

Missing SemiColons:

```
.bad {  
  color: red  
background-color: yellow;  
}
```

Unclosed Braces:

```
.bad {  
padding: 20px;  
margin: 10px;
```

CSS COLORS

CSS RGB Colors :

An RGB color value represents RED, GREEN, and BLUE light sources.

<code>rgb(255, 0, 0)</code>	<code>rgb(0, 0, 255)</code>
<code>rgb(60, 179, 113)</code>	<code>rgb(238, 130, 238)</code>
<code>rgb(255, 165, 0)</code>	<code>rgb(106, 90, 205)</code>

CSS HEX Colors :

A hexadecimal color is specified with: #RRGGBB.

<code>#ff0000</code>	<code>#0000ff</code>
<code>#3cb371</code>	<code>#ee82ee</code>
<code>#ffa500</code>	<code>#6a5acd</code>

CSS HSL Colors :

HSL stands for hue, saturation, and lightness.

<code>hsl(0, 100%, 50%)</code>	<code>hsl(240, 100%, 50%)</code>
<code>hsl(147, 50%, 47%)</code>	<code>hsl(300, 76%, 72%)</code>
<code>hsl(39, 100%, 50%)</code>	<code>hsl(248, 53%, 58%)</code>

CSS TEXT

CSS Colors :

```
body {  
  background-color: lightgrey;  
  color: blue;  
}
```

CSS Text Alignment :

```
h1 {  
  text-align: center;  
}
```

- `left` - Aligns the text to the left
- `right` - Aligns the text to the right
- `center` - Centers the text
- `justify` - Stretches the lines so that each line has equal width

CSS Text Decoration :

```
h1 {  
  text-decoration-line: overline;  
}
```

- `none` - Default value. Displays no decoration line
- `underline` - The decoration line is displayed under the text
- `overline` - The decoration line is displayed over the text
- `line-through` - The decoration line is displayed through the text

CSS TEXT

CSS Text Decoration Styles :

```
h1 {  
  text-decoration-line: underline;  
  text-decoration-style: solid;  
}
```

- `solid` - Default value. Single line
- `double` - Double line
- `dotted` - Dotted line
- `dashed` - Dashed line
- `wavy` - Wavy line

CSS Text Transformation :

```
p.uppercase {  
  text-transform: uppercase;  
}
```

- `none` - No transformation. Text renders as it is
- `capitalize` - Transforms the first character of each word to uppercase
- `uppercase` - Transforms all characters to uppercase
- `lowercase` - Transforms all characters to lowercase

CSS Text Spacing :

```
p {  
  text-indent: 50px;  
}
```

- `text-indent`
- `letter-spacing`
- `line-height`
- `word-spacing`
- `white-space`

CSS BACKGROUNDS

Background-Color :

```
body {  
  background-color: lightblue;  
}
```

Background-Image :

```
body {  
  background-image: url("paper.gif");  
}
```

Background-Repeat :

```
body {  
  background-image: url("img_tree.png");  
  background-repeat: no-repeat;  
}
```

CSS BACKGROUNDS

Background-Attachment :

```
body {  
  background-image: url("img_tree.png");  
  background-attachment: scroll; /*fixed*/  
}
```

Background-Position :

```
body {  
  background-image: url("img_tree.png");  
  background-repeat: no-repeat;  
  background-position: right top;  
}
```

Background : (Short-Hand)

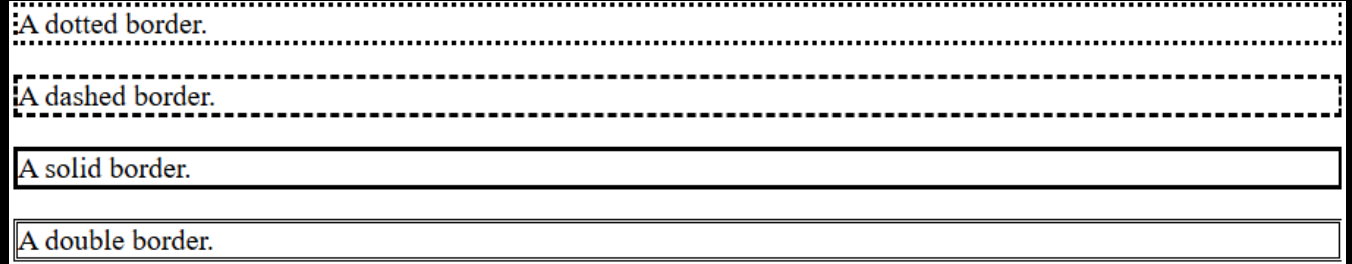
```
body {  
  background: #ffffff url("img_tree.png")  
  no-repeat right top;  
}
```

- background-color
- background-image
- background-position
- background-size
- background-repeat
- background-attachment
- background-origin
- background-clip

CSS BORDERS

Border-Style :

```
.dotted {border-style: dotted;}  
.dashed {border-style: dashed;}  
.solid {border-style: solid;}  
.double {border-style: double;}
```



Border-Width :

```
p.one {  
  border-style: solid;  
  border-width: 5px 20px; /* 5px top and bottom, 20px on the sides */  
}
```

Border-Color :

```
p.one {  
  border-style: solid;  
  border-color: red;  
}
```

CSS BORDERS

Border-Sides:

```
p {  
  border-top-style: dotted;  
  border-right-style: solid;  
  border-bottom-style: dotted;  
  border-left-style: solid;  
}
```

different border styles.

Border-Shorthand :

```
p {  
  border: 5px solid red;  
}
```

Rounded-Borders :

```
p {  
  border: 2px solid red;  
  border-radius: 5px;  
}
```

CSS MARGIN & PADDING

Margin :

```
p {  
  margin-top: 100px;  
  margin-bottom: 100px;  
  margin-right: 150px;  
  margin-left: 80px;  
}
```

- **margin: 25px 50px 75px 100px;**
 - top margin is 25px
 - right margin is 50px
 - bottom margin is 75px
 - left margin is 100px

Padding :

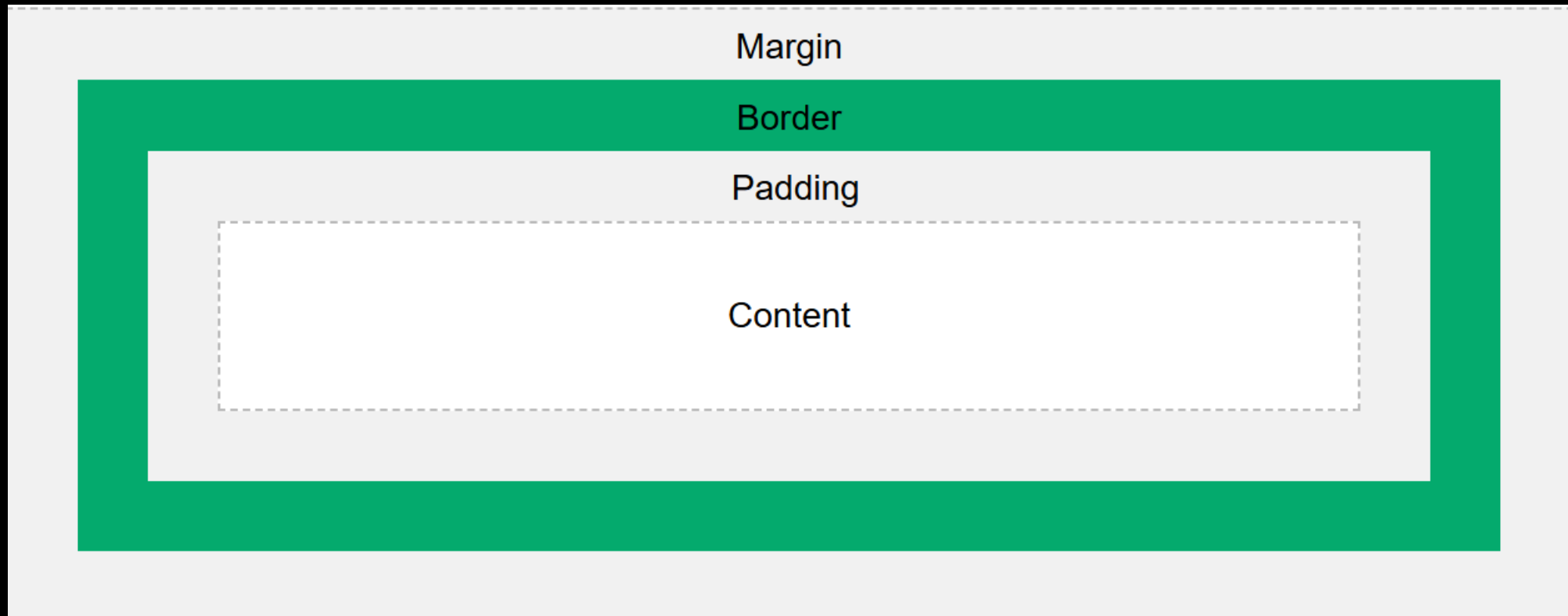
```
div {  
  padding-top: 50px;  
  padding-right: 30px;  
  padding-bottom: 50px;  
  padding-left: 80px;  
}
```

- **padding: 25px 50px 75px 100px;**
 - top padding is 25px
 - right padding is 50px
 - bottom padding is 75px
 - left padding is 100px

The box-sizing property can have the following values:

- **content-box** - This is default. The width and height properties includes only the content (border and padding are not included)
- **border-box** - The width and height properties includes content, padding and border

CSS BOX MODEL



- **Content** - The content of the box, where text and images appear
- **Padding** - Clears an area around the content. The padding is transparent
- **Border** - A border that goes around the padding and content
- **Margin** - Clears an area outside the border. The margin is transparent

CSS BOX MODEL

```
div {  
  width: 320px;  
  height: 50px;  
  padding: 10px;  
  border: 5px solid gray;  
  margin: 0;  
  box-sizing: content-box;  
}
```

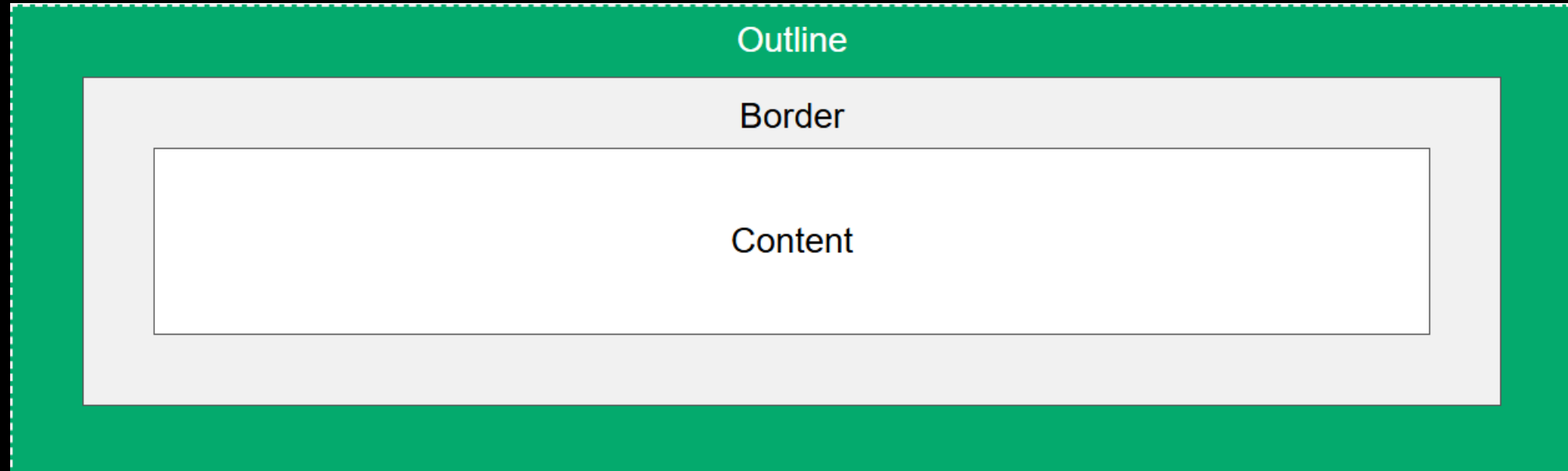


```
div {  
  width: 320px;  
  height: 50px;  
  padding: 10px;  
  border: 5px solid gray;  
  margin: 0;  
  box-sizing: border-box;  
}
```



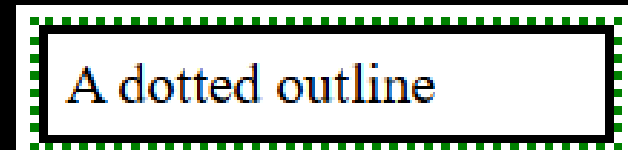
Width & Height includes content , padding & border

CSS OUTLINE



Outline Style :

```
p.dotted {  
  outline-style: dotted;  
  border-style: solid;  
}
```



CSS OUTLINE

Outline Color :

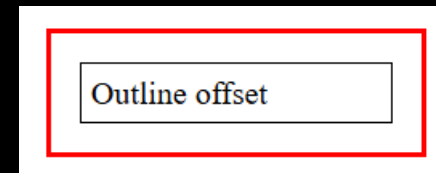
```
p.ex1 {  
  outline-style: solid;  
  outline-color: red;  
}
```

Outline Width :

```
p.ex1 {  
  outline-width: thin; /*Medium , Thick */  
}
```

Outline Offset:

```
p {  
  border: 1px solid black;  
  outline: 3px solid red;  
  outline-offset: 15px;  
}
```



Outline :

```
p.ex1 {outline: 7px solid yellow;}
```

CSS *Display* property

Value	Description
inline	Displays an element as an inline element
block	Displays an element as a block element
contents	Makes the container disappear, making its child elements children of the element the next level up in the DOM
flex	Displays an element as a block-level flex container
grid	Displays an element as a block-level grid container
inline-block	Displays an element as an inline-level block container. The element itself is formatted as an inline element, but you can apply height, width, padding, and margin values
none	The element is completely hidden from the document flow (does not take up any space).

HTML File

```
<p class="ex1">  
  <span>Text 1</span>  
  <span>Text 2</span>  
  <span>Text 3</span>  
</p>
```

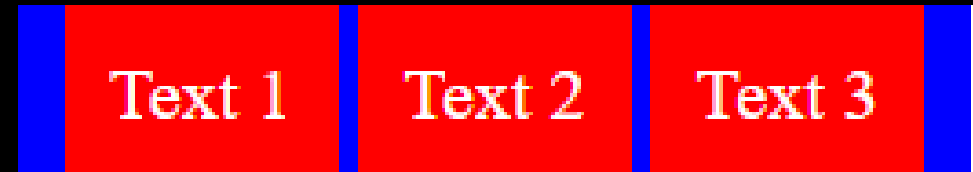
CSS *Display* property

Display : none

p.ex1 {display: none;}

Display : inline

p.ex1 {display: inline;}



Display : block

p.ex1 {display: block;}



CSS *Display* property

Display : inline-block

```
p.ex1 {display: inline-block;}
```



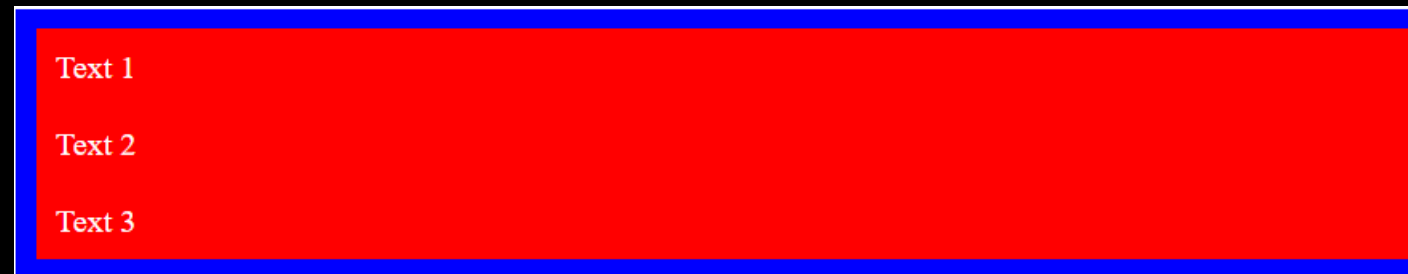
Display : flex

```
p.ex1 {display: flex;}
```



Display : grid

```
p.ex1 {display: grid;}
```



CSS Flex Container Properties

The flex container element can have the following properties:

- `display` - Must be set to `flex` or `inline-flex`
- `flex-direction` - Sets the display-direction of flex items
- `flex-wrap` - Specifies whether the flex items should wrap or not
- `flex-flow` - Shorthand property for `flex-direction` and `flex-wrap`
- `justify-content` - Aligns the flex items when they do not use all available space on the main-axis (horizontally)
- `align-items` - Aligns the flex items when they do not use all available space on the cross-axis (vertically)
- `align-content` - Aligns the flex lines when there is extra space in the cross axis and flex items wrap

Flex-Direction :

```
.flex-container{  
  display: flex;  
  flex-direction: row;  
}
```

- `row` (default)
- `column`
- `row-reverse`
- `column-reverse`

CSS Flex Container Properties

Flex-Wrap :

```
.flex-container {  
  display: flex;  
  flex-wrap: nowrap;  
}
```

- `nowrap` (default)
- `wrap`
- `wrap-reverse`

Flex-Flow :

```
.flex-container {  
  display: flex;  
  flex-flow: row wrap;  
}
```

Justify-Content :

```
.flex-container {  
  display: flex;  
  justify-content: center;  
}
```

- `center`
- `flex-start` (default)
- `flex-end`
- `space-around`
- `space-between`
- `space-evenly`

CSS Flex Container Properties

Align-items :

```
.flex-container {  
  display: flex;  
  height: 200px;  
  align-items: center;  
}
```

- `normal` (default)
- `stretch`
- `center`
- `flex-start`
- `flex-end`
- `baseline`

Align-Content :

```
.flex-container {  
  display: flex;  
  height: 400px;  
  flex-wrap: wrap;  
  align-content: center;  
}
```

- `stretch` (default)
- `center`
- `flex-start`
- `flex-end`
- `space-between`
- `space-around`
- `space-evenly`

```
.flex-container {  
  display: flex;  
  justify-content: center;  
  align-items: center;  
}
```

CSS Flex Items Properties

Flex items can have the following properties:

- **order** - Specifies the display order of the flex items inside the flex container
- **flex-grow** - Specifies how much a flex item will grow relative to the rest of the flex items
- **flex-shrink** - Specifies how much a flex item will shrink relative to the rest of the flex items
- **flex-basis** - Specifies the initial length of a flex item
- **flex** - Shorthand property for **flex-grow**, **flex-shrink**, and **flex-basis**
- **align-self** - Specifies the alignment for the flex item inside the flex container

Order :

```
<div class="flex-container">  
  <div style="order: 3">1</div>  
  <div style="order: 2">2</div>  
  <div style="order: 4">3</div>  
  <div style="order: 1">4</div>  
</div>
```



CSS GRID

CSS Grid Tracks :

- `grid-template-columns` - Defines the number and width of the columns in the grid
- `grid-template-rows` - Defines the number and height of the rows in the grid
- `grid-template-areas` - Defines how to display columns and rows, using named grid items

```
.container{  
  display: grid;  
  grid-template-columns: 80px 200px auto;  
}
```

1	2	3
4	5	6

```
.container{  
  display: grid;  
  grid-template-columns: repeat(3, 1fr);  
}
```

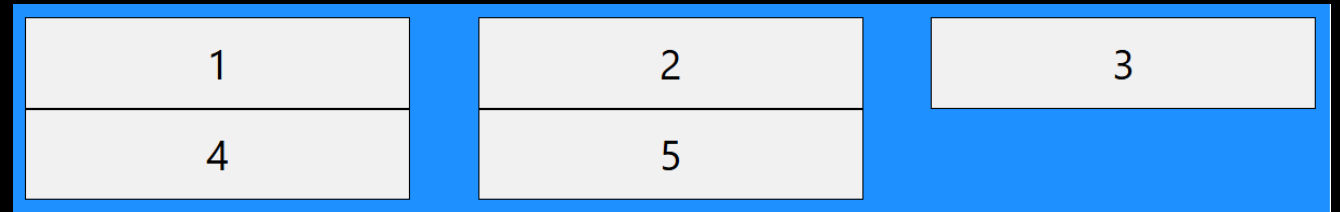
1	2	3
4	5	6

CSS GRID

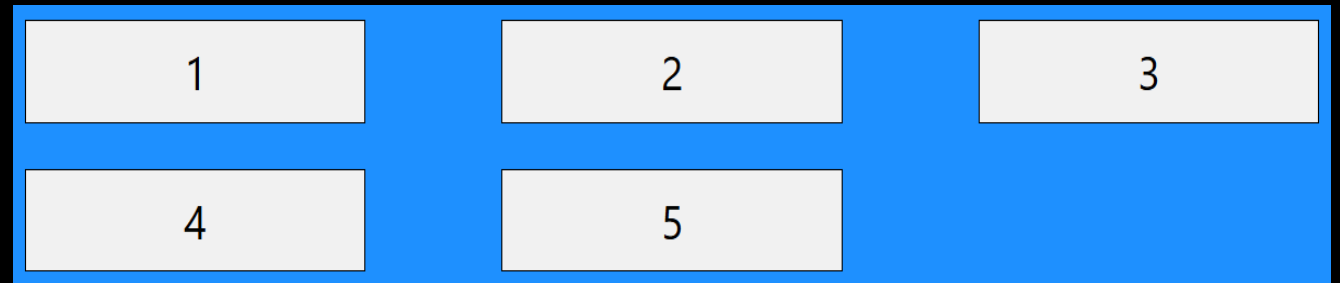
CSS Grid Gaps :

- `column-gap` - Specifies the gap between grid columns
- `row-gap` - Specifies the gap between grid rows
- `gap` - Shorthand property for `row-gap` and `column-gap`

```
.container{  
  display: grid;  
  column-gap: 50px;  
}
```



```
.container{  
  display: grid;  
  gap: 30px 100px;  
}
```



CSS GRID

CSS Grid Tracks :

- `grid-template-columns` - Defines the number and width of the columns in the grid
- `grid-template-rows` - Defines the number and height of the rows in the grid
- `grid-template-areas` - Defines how to display columns and rows, using named grid items

```
.container{  
  display: grid;  
  grid-template-columns: 80px 200px auto;  
}
```

1	2	3
4	5	6

```
.container{  
  display: grid;  
  grid-template-columns: repeat(3, 1fr);  
}
```

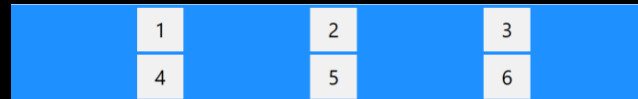
1	2	3
4	5	6

CSS GRID

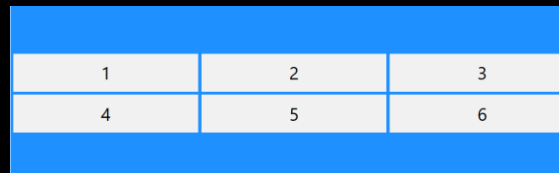
CSS Grid Justify & Align :

- `justify-content` - Aligns the grid content when it does not use all available space on the main-axis (horizontally)
- `align-content` - Aligns the grid content when it does not use all available space on the cross-axis (vertically)
- `place-content` - Shorthand property for `align-content` and `justify-content`

```
.container{  
  display: grid;  
  justify-content: space-evenly;  
}
```



```
.container{  
  display: grid;  
  height: 300px;  
  align-content: center;  
}
```



- `space-evenly`
- `space-around`
- `space-between`
- `center`
- `start`
- `end`

CSS Grid Items Properties

- `grid-column-start` - Specifies on which column-line the grid item will start
- `grid-column-end` - Specifies on which column-line the grid item will end
- `grid-column` - Shorthand property for `grid-column-start` and `grid-column-end`
- `grid-row-start` - Specifies on which row-line the grid item will start
- `grid-row-end` - Specifies on which row-line the grid item will end
- `grid-row` - Shorthand property for `grid-row-start` and `grid-row-end`

Order :

```
.item1 {order: 3;}  
.item2 {order: 6;}  
.item3 {order: 1;}  
.item4 {order: 2;}  
.item5 {order: 4;}  
.item6 {order: 5;}
```

3	4	1
5	6	2

CSS POSITION

- **static** - This is default. Element is positioned according to the normal document flow
- **relative** - Element is positioned relative to its normal position in the document flow
- **fixed** - Element is positioned relative to the viewport
- **absolute** - Element is positioned relative to the nearest positioned ancestor
- **sticky** - Element toggles between a relative and fixed position, depending on the scroll position

Relative / Absolute :

```
div.relative {  
  position: relative;  
  width: 400px;  
  height: 200px;  
  border: 3px solid green;  
}
```

```
div.absolute {  
  position: absolute;  
  top: 80px;  
  right: 0;  
  border: 3px solid red;  
}
```

This div element has position: relative;

This div element has position:
absolute;

CSS LISTS

List-Style-Type :

```
ul.a {list-style-type: circle;}  
ul.b {list-style-type: disc;}  
ul.c {list-style-type: square;}
```

```
ol.d {list-style-type: upper-roman;}  
ol.e {list-style-type: lower-roman;}  
ol.f {list-style-type: lower-alpha;}  
ol.g {list-style-type: decimal;}
```

List-Style-Image :

```
ul {  
  list-style-image: url('sqpurple.gif');  
  list-style-type: square;  
}
```

List-Style-Position :

```
ul.a {  
  list-style-position: outside;  
}  
  
ul.b {  
  list-style-position: inside;  
}
```

- Coffee
- Tea
- Coca-cola

- Coffee
- Tea
- Coca-cola

CSS TABLES

Table-Borders :

```
table, th, td {  
  border: 1px solid;  
}
```

The **border** property is a shorthand property for:

- **border-width** - sets the width of the border
- **border-style** - sets the style of the border (required)
- **border-color** - sets the color of the border

Collapse Table-Borders :

```
table {  
  border-collapse: collapse;  
}
```

- **separate** - Default value. Borders are separated; each cell will display its own borders
- **collapse** - Borders are collapsed into a single border when possible

Borders Spacing :

```
table {  
  border-collapse: separate;  
  border-spacing: 15px;  
}
```

Firstname	Lastname
Peter	Griffin
Lois	Griffin

Note: This property works only when border-collapse is set to "separate".

CSS UNITS

Absolute-Units :

Unit	Name	Description
px	Pixels	The most used absolute unit for screens (1px = 1/96th of 1in)
cm	Centimeters	Primarily used in print stylesheet
mm	Millimeters	Primarily used in print stylesheet
in	Inches	Primarily used in print stylesheet (1in = 96px = 2.54cm)
pt	Points	A typographical unit (1pt = 1/72 of 1in)
pc	Picas	A print unit (1pc = 12 pt)

- Absolute units are fixed, and the length expressed in any of these will appear exactly that size.
- Absolute units do NOT change when the screen size change

Relative-Units :

Unit	Description
em	Relative to the font-size of the parent element
rem	Relative to the font-size of the root HTML element
vw	Relative to 1% of the width of the viewport*. 100vw = full width of the viewport
vh	Relative to 1% of the height of the viewport*. 100vh = full height of the viewport
vmin	Relative to 1% of viewport's* smaller dimension
vmax	Relative to 1% of viewport's* larger dimension
%	Relative to the size of the parent element
fr	A fractional unit. 1fr equals 1 part of the available space
ch	Relative to width of the "0" (zero) character

- Relative units specify a length relative to another length property (like parent element, root element, or viewport).

CSS Pseudo-Classes

Interactive Pseudo-Classes :

Interactive pseudo-classes apply styles based on **user interaction**:

- `:hover` - When mouse is over an element
- `:focus` - When an element has focus
- `:active` - When an element is being activated
- `:link` - Unvisited links
- `:visited` - Visited links

```
/* unvisited link */  
a:link {  
    color: #FF0000;  
}
```

```
/* visited link */  
a:visited {  
    color: #00FF00;  
}
```

```
/* mouse over link */  
a:hover {  
    color: #FF00FF;  
}
```

```
/* selected link */  
a:active {  
    color: #0000FF;  
}
```

Structural Pseudo-Classes :

```
p:first-child {  
    color: blue;  
}
```

This is some text.
This is some text.

Structural pseudo-classes select elements based on **their position in the document tree**:

- `:first-child` - First child of a parent
- `:last-child` - Last child of a parent
- `:nth-child(n)` - The nth child of a parent
- `:lang()` - Elements with a specific language

CSS Pseudo-Elements

Text Pseudo-Elements :

Text pseudo-elements style specific parts of text content:

- `::first-line` - Style the first line of text
- `::first-letter` - Style the first letter of text

```
p::first-letter {  
  color: red;  
  font-size: xx-large;  
}
```

```
p::first-line {  
  color: blue;  
  font-variant: small-caps;  
}
```

YOU CAN COMBINE THE `::FIRST-LETTER` AND `::FIRST-LINE` PSEUDO-ELEMENTS TO ADD A SPECIAL EFFECT TO THE first letter and the first line of a text!

Content Pseudo-Elements :

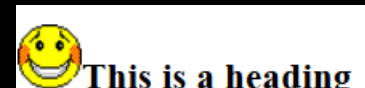
Content pseudo-elements insert or style generated content:

- `::before` - Insert content before an element
- `::after` - Insert content after an element
- `::marker` - Style list item markers
- `::selection` - Style user-selected text
- `::backdrop` - Style dialog backdrop

CSS Pseudo-Elements

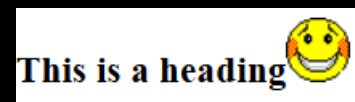
::before

```
h3::before {  
  content: url(smiley.gif);  
}
```



::after

```
h3::after {  
  content: url(smiley.gif);  
}
```



::marker

```
::marker {  
  color: red;  
  font-size: 23px;  
}
```

- Coffee
- Tea
- Milk

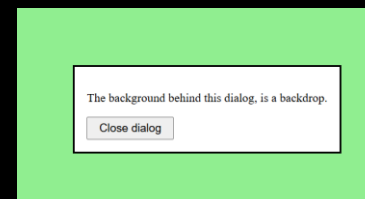
::selection

```
::selection {  
  color: red;  
  background: yellow;  
}
```

This is a paragraph.

::backdrop

```
dialog::backdrop {  
  background-color: lightgreen;  
}
```



Content Pseudo-Elements

CSS Overflow

The `overflow` property has the following values:

- `visible` - Default. The overflow is not clipped. The content renders outside the element's box
- `hidden` - The overflow is clipped, and the rest of the content is hidden
- `scroll` - Scrollbars are added. User must scroll to see all content
- `auto` - Similar to `scroll`, but adds scrollbars only when necessary

CSS Overflow : Visible

You can use the overflow property when you want to have better control of the layout. The overflow property specifies what happens if content overflows an element's box.

CSS Overflow : Hidden

You can use the overflow property when you want to have better control of the layout. The overflow

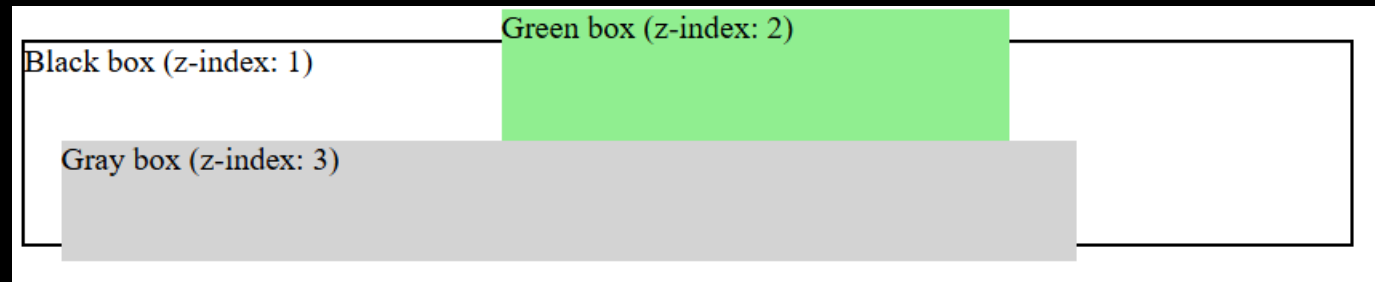
CSS Overflow : Scroll

You can use the overflow property when you want to have better control of the layout.

CSS Overflow : Auto

You can use the overflow property when you want to have better control of the layout. The overflow property

CSS Z-Index



The z-index property specifies the stack order of positioned elements.

```
img {  
  position: absolute;  
  left: 0px;  
  top: 0px;  
  z-index: -1;  
}
```

CSS Float

Float



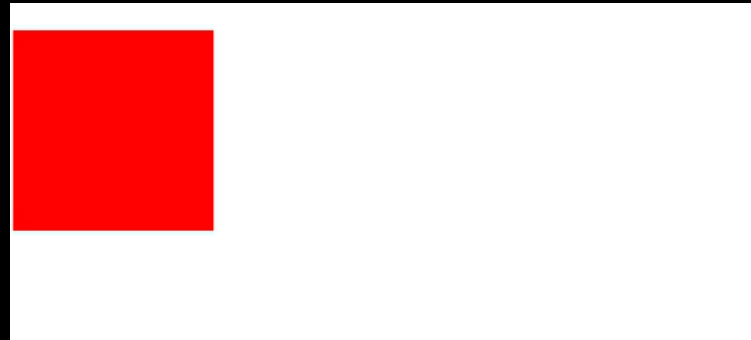
```
img {  
  float: right;  
  margin-left: 15px;  
  width: 200px;  
  height: 200px;  
}
```

```
<div>  
<p>  
    
  Float</p>  
</div>
```

CSS TRANSITIONS

CSS transitions allows you to change property values smoothly, over a given duration.

```
div{  
  width: 100px;  
  height: 100px;  
  background-color: red;  
  transition: width 2s;  
}  
  
div:hover{  
  width: 300px;  
}
```



The CSS `transition` property is a shorthand property for:

- `transition-property`
- `transition-duration`
- `transition-timing-function`
- `transition-delay`

```
div{  
  width: 100px;  
  height: 100px;  
  background-color: red;  
  transition: width 2s, height  
    4s, background-color 3s;  
}
```

CSS Transition

CSS Transition timing Function

```
div {  
  transition: width 2s linear 1s;  
}
```



```
div {  
  transition-property: width;  
  transition-duration: 2s;  
  transition-timing-  
function: linear;  
  transition-delay: 1s;  
}
```

- **ease** - transition will start slow, then go fast, and end slow (this is default)
- **linear** - transition will keep the same speed from start to end
- **ease-in** - transition will start slow
- **ease-out** - transition will end slow
- **ease-in-out** - transition will have a slow start and end
- **cubic-bezier(n,n,n,n)** - lets you define your own values in a cubic-bezier function

Ex : (Transform)

```
button {  
  transition: background-color 1s ease-out,  
  transform 1s ease-out;  
}
```

CSS Animations

CSS allows animation of HTML elements without using JavaScript!

```
/* The animation code */
@keyframes myAnimation {
  from {background-color: red;}
  to {background-color: yellow;}
}

/* The element to apply the animation to */
div {
  width: 100px;
  height: 100px;
  background-color: red;
  animation-name: myAnimation;
  animation-duration: 4s;
}
```

```
@keyframes myAnimation {
  0% {background-color:red; left:0px; top:0px;}
  25% {background-color:yellow; left:200px; top:0px;}
  50% {background-color:blue; left:200px; top:200px;}
  75% {background-color:green; left:0px; top:200px;}
  100% {background-color:red; left:0px; top:0px;}
}

div {
  width: 100px;
  height: 100px;
  position: relative;
  background-color: red;
  animation-name: myAnimation;
  animation-duration: 4s;
}
```

CSS Animations

Animation Properties

- The **animation-iteration-count** property specifies the number of times an animation should run.
- The **animation-timing-function** property specifies the speed curve of the animation.
- The **animation-direction** property specifies whether an animation should be played forwards, backwards or in alternate cycles.

- **normal** - The animation is played as normal (forwards). This is default
- **reverse** - The animation is played in reverse direction (backwards)
- **alternate** - The animation is played forwards first, then backwards
- **alternate-reverse** - The animation is played backwards first, then forwards

- The **animation-fill-mode** property specifies a style for the target element when the animation is not playing (before it starts, after it ends, or both).

- **none** - Default value. Animation will not apply any styles to the element before or after it is executing
- **forwards** - The element will retain the style values that is set by the last keyframe (depends on animation-direction and animation-iteration-count)
- **backwards** - The element will get the style values that is set by the first keyframe (depends on animation-direction), and retain this during the animation-delay period
- **both** - The animation will follow the rules for both forwards and backwards, extending the animation properties in both directions

CSS Animations

```
div {  
  animation: myAnimation 5s linear 2s infinite alternate;  
}
```



```
div {  
  animation-name: myAnimation;  
  animation-duration: 5s;  
  animation-timing-function: linear;  
  animation-delay: 2s;  
  animation-iteration-count: infinite;  
  animation-direction: alternate;  
}
```

Property	Description
@keyframes	Specifies the animation code
animation	A shorthand property for setting all the animation properties
animation-delay	Specifies a delay for the start of an animation
animation-direction	Specifies whether an animation should be played forwards, backwards or in alternate cycles
animation-duration	Specifies how long time an animation should take to complete one cycle
animation-fill-mode	Specifies a style for the element when the animation is not playing (before it starts, after it ends, or both)
animation-iteration-count	Specifies the number of times an animation should be played
animation-name	Specifies the name of the @keyframes animation

CSS Media Queries

CSS media queries allow you to apply styles based on the characteristics of a device or the environment displaying the web page.

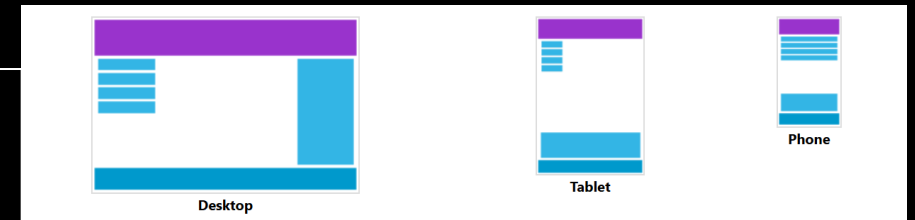
```
/* Extra small devices (phones, 600px and down) */  
@media only screen and (max-width: 600px) {...}
```

```
/* Small devices (portrait tablets and large phones, 600px and up) */  
@media only screen and (min-width: 600px) {...}
```

```
/* Medium devices (landscape tablets, 768px and up) */  
@media only screen and (min-width: 768px) {...}
```

```
/* Large devices (laptops/desktops, 992px and up) */  
@media only screen and (min-width: 992px) {...}
```

```
/* Extra large devices (large laptops and desktops, 1200px and up) */  
@media only screen and (min-width: 1200px) {...}
```



CSS Media Queries

Media Queries for Screen Orientation

```
@media only screen and (orientation: landscape) {  
  body {  
    background-color: lightblue;  
  }  
}
```

Media Queries for User Preferences

```
@media (prefers-reduced-motion: reduce) {  
  * {  
    animation: none !important;  
    transition: none !important;  
  }  
}
```

Hide Elements with Media Queries

```
/* Hide element if the viewport width is 600px or less */  
@media screen and (max-width: 600px) {  
  #div1 {  
    display: none;  
  }  
}
```

CSS Specificity

Selector	Example	Description	Weight
Inline styles	<code><h1 style="color: pink;"></code>	Highest priority, will override all other selectors	
Id selectors	<code>#navbar</code>	Second highest priority	1-0-0
Classes, attribute selectors and pseudo-classes	<code>.test</code> , <code>[type="text"]</code> , <code>:hover</code>	Third highest priority	0-1-0
Elements and pseudo-elements	<code>h1</code> , <code>::before</code> , <code>::after</code>	Low priority	0-0-1
Universal selector and <code>:where()</code>	<code>*</code> , <code>where()</code>	No priority	0-0-0

The weight notation uses three numbers (X-Y-Z):

- **X** - Number of ID selectors
- **Y** - Number of class selectors, attribute selectors, and pseudo-classes
- **Z** - Number of element selectors and pseudo-elements

The selector with the higher leftmost number wins. If equal, compare the next number, and so on.

Prefer This



[Specificity Calculator](#)

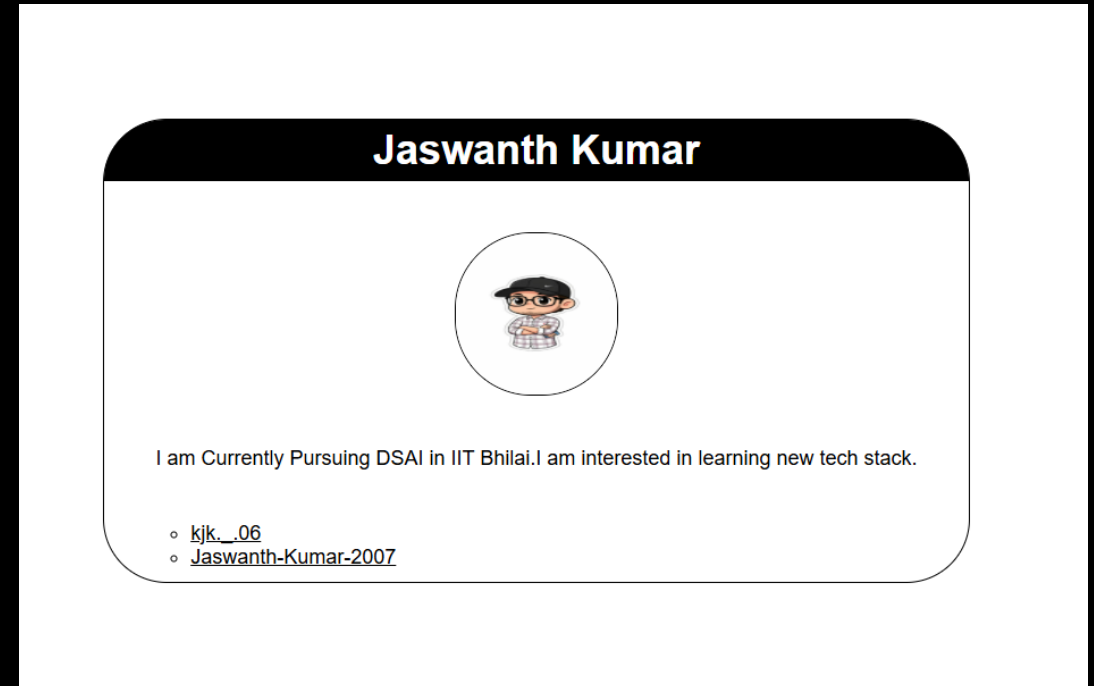
Project for Session

"Build a personal profile page in pure HTML/CSS:

- A header with your name + photo placeholder
- A short bio section
- A links section

Push it to a public GitHub repo!"

Add CSS to your Already Made HTML Profile Page



Useful Resources for CSS

- [CSS Validator](#)
- [Specificity Calculator](#)
- [Browse Fonts - Google Fonts](#)
- [SYMBL \(🐣🐣\) Symbols, Emojis, Characters, Scripts, Alphabets, Hieroglyphs and the entire Unicode](#)
- [Anchoreum - A game for learning CSS anchor positioning](#)
- [Flexbox Froggy - A game for learning CSS flexbox](#)
- [Grid Garden - A game for learning CSS grid](#)

THANK YOU



Meet Again in Next Session.... 😊